CS243X
THE UNIVERSITY OF WARWICK
Second Year Examinations, January 2013
Data Structures and Algorithms & Algorithm Design

- **Time allowed: 1.5 hours per SECTION (combined 3 hours).**
- Answer **TWO** questions **from each section** for which you are registered.
- Use **separate answerbooks** for answering questions from the two sections. In other words, put all of your answers to questions from section A into one answerbook, and put all of your answers to questions from section B into the other answerbook. Make it clear on the cover of each answerbook which section (A or B) it is for, and which two questions from the respective section you have selected.
- Calculators are not required and not permitted.

Section A: Data Structures and Algorithms (CS 243)

- **Time allowed for this section: 1.5 hours.**
- Answer **TWO** questions from this section. Make it clear on the cover of the answerbook which two questions you have selected.
- Each question is worth a total of 50 marks. The nominal credit mark assigned to each part of a question is indicated by a bold figure next to that part.

---

1. The Brewed Awakening Co-operative (BAcoop) is planning to open a series of cafés at service stations on the M1 motorway (which goes north starting from London). There are n possible locations and they are numbered 1, 2, ..., n, in the order of increasing distance from London. The distance (in miles) of location i from London is $D[i]$ and hence $D[1] < D[2] < \cdots < D[n]$.

   The BAcoop has a contract with the Highways Agency which requires that the gaps between their consecutive cafés to be at least $G > 0$ miles (the "minimum-gaps" contract).

   (a) **[8 marks]** Consider the following locations and their distances from London:

Location i	1	2	3	4	5	6	7	8
Distance from London $D[i]$	0	10	25	30	55	75	80	100

   Note that the set $\{ 1, 4, 7 \}$ satisfies the minimum-gaps contract for $G = 25$.

   Give a set of five locations that satisfies the minimum-gaps contract for $G = 20$.

   What is the largest size of a set of locations that satisfies the minimum-gaps contract for $G = 30$? Give such a set.

   (b) **[12 marks]** Design an efficient algorithm that—given a table $D[1..n]$ of distances from London and a number $G > 0$—computes a set of locations of largest size that satisfies the minimum-gaps contract for the given value of G.

   Justify the correctness of your algorithm. What is the asymptotic running time of your algorithm?

   Hint: Relate this problem to the interval scheduling problem.

   Assume that BAcoop knows not only the distances of service stations from London, but also, for each location i, the annual profit $P[i]$ of running a café at location i.

   (c) **[12 marks]** Consider the following table of locations, their distances from London and annual profits:

Location i	1	2	3	4	5	6	7
Distance from London $D[i]$	0	20	30	40	60	70	100
Expected annual profit $P[i]$	2	1	4	3	3	4	4

   What is the total annual profit earned by choosing the set $\{ 1, 4, 7 \}$ of locations?

   For each $i = 1, 2, \ldots, n$, we define $M[i]$ to be the maximum total annual profit that can be earned by choosing a subset of locations 1, 2, ..., i that is consistent with the minimum-gaps contract for a given value of G.

   For example, if $G = 20$ then $M[1] = P[1] = 2$ and $M[2] = P[1] + P[2] = 3$.

   Compute the values $M[3], M[4], M[5], M[6]$, and $M[7]$ for $G = 30$. Justify your answer for $M[4]$ and for $M[5]$.

   (d) **[18 marks]** Design an efficient algorithm that—given the tables $D[1..n]$ of distances from London, $P[1..n]$ of profits, and a number G as inputs—computes the maximum total annual profit BAcoop can achieve under the minimum-gaps contract.

   Justify the correctness of your algorithm, and analyse its asymptotic running time.

2. We model a set of cities and highways as an undirected weighted graph $G = (V, E, \ell)$, where the vertices V represent cities, edges E represent highways connecting cities, and for every undirected edge $e = \{v, w\} \in E$ the number $\ell[e]$ denotes the number of litres of fuel that your motorcycle needs in order to ride the distance between cities v and w.

   There are fuel stations in every city but none on the highways between the cities. Therefore, if the capacity of your motorcycle's fuel tank is L litres, then you can only follow a path (a sequence of adjacent highways), if for every highway e on this path, we have $L \geq \ell[e]$, because you can only refuel in the cities.

   Let $n = |V|$ be the number of cities, and let $m = |E|$ be the number of highways.

   (a) **[6 marks]** Consider an undirected weighted graph $G = (V, E, \ell)$ with four cities $V = \{A, B, C, D\}$ and the numbers $\ell[e]$ defined for every highway $e \in E$ by the following table:

e	$\{A, B\}$	$\{A, C\}$	$\{B, C\}$	$\{B, D\}$	$\{C, D\}$
$\ell[e]$	8	4	6	5	9

   where $E = \{\, \{A, B\}, \{A, C\}, \{B, C\}, \{B, D\}, \{C, D\} \,\}$.

   Draw the graph G and give a path from city A to city D that can be followed by a motorcycle whose fuel tank capacity is 7 litres.

   (b) **[14 marks]** Design an efficient algorithm that—given an undirected weighted graph $G = (V, E, \ell)$ modelling a set of cities and highways, two cities $s, t \in V$, and a capacity L of your motorcycle's fuel tank as inputs—answers whether your motorcycle can reach city t from city s or not.

   Justify correctness of your algorithm. For full credit your algorithm should run in time $O(n + m)$.

   (c) **[6 marks]** For the example graph from part (a), what is the smallest fuel tank capacity of a motorcycle that can:

   i. reach city B from city A?
   ii. reach city C from city A?
   iii. reach city D from city A?

   In each case, give a path that can be followed with this smallest fuel tank capacity.

   (d) **[18 marks]** Design an efficient algorithm that—given an undirected weighted graph $G = (V, E, \ell)$ modelling a set of cities and highways, and a city $s \in V$ as inputs—computes, for every other city $v \in V$, the smallest fuel tank capacity $M[v]$ that your motorcycle needs in order to be able to reach city v from city s.

   Prove that your algorithm is correct and analyse its running time. For full credit your algorithm should run in time $O(m \log n)$.

   (e) **[6 marks]** Illustrate the behaviour of your algorithm from part (d) on the example from part (a).

3. (a) **[10 marks]** Suppose you are choosing between the following three algorithms:

   - Algorithm A solves a problem by dividing it into five subproblems of half the size, recursively solving each subproblem, and then combining the solutions in linear time.
   - Algorithm B solves a problem of size n by dividing it into nine subproblems of size $n/3$, recursively solving each subproblem, and then combining the solutions in $\Theta(n^2)$ time.
   - Algorithm C solves a problem of size n by recursively solving two subproblems of size $n-1$ and then combining the solutions in constant time.

   Write down the recurrences for the asymptotic running times $T_A(n)$, $T_B(n)$, and $T_C(n)$ of the algorithms A, B, and C, respectively.

   (b) **[15 marks]** Determine the asymptotic running times of the algorithms A, B, and C from part (a) by solving the recurrences that you have provided. Justify your answers.

   Which algorithm is asymptotically most efficient? Which is asymptotically least efficient?

   (c) **[25 marks]** Let $A[1..n]$ be a sorted array of n distinct integers. In other words, for every $i = 1, 2, \ldots, n-1, n$, we have that $A[i] \in \{\ldots, -2, -1, 0, 1, 2, \ldots\}$ and

$$A[1] < A[2] < \cdots < A[n-1] < A[n].$$

   You are supposed to find out whether there is an index i, such that $A[i] = i$. Give a divide-and-conquer algorithm which finds and returns such an index i if there is one, or returns "NO" if $A[i] \neq i$ for all $i = 1, 2, \ldots, n$.

   Justify the correctness of your algorithm.

   Characterize the running time of your algorithm using a recurrence, and solve the recurrence to obtain an asymptotic upper running time bound.

   A trivial brute-force solution with $O(n)$ running time is not acceptable. Your algorithm should run in $O(\log n)$ time to attract significant credit.

End (of CS 243; pages 2 – 5)

Section B: Algorithm Design (CS 244)

- **Time allowed for this section: 1.5 hours.**
- Answer **TWO** questions from this section. Make it clear on the cover of the answerbook which two questions you have selected.
- Each question is worth a total of 50 marks. The nominal credit mark assigned to each part of a question is indicated by a bold figure next to that part.

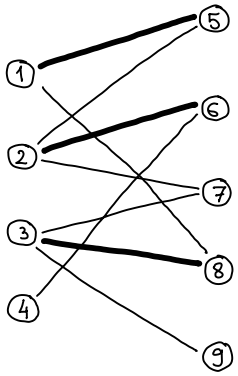
---

1. It is possible to find a maximum-size matching in a bipartite graph using a polynomial-time reduction to the maximum flow problem. In this question you are asked to develop a direct algorithm for the problem.

   Consider a bipartite graph $G = (L \cup R, E)$; each edge has one endpoint in L and the other one in R, and $V = L \cup R$ is the set of all vertices. Let $n = |V|$ and $m = |E|$ be the numbers of vertices and edges in graph G, respectively.

   Let $M \subseteq E$ be a matching in the bipartite graph G. A vertex is said to be *covered* by the matching M if it is the endpoint of one of the edges in M. An *alternating path* with respect to a matching M is a path in graph G consisting of odd number of edges that starts and ends with a non-covered vertex, and whose edges alternate between $E \setminus M$ and M.

   Consider the following example of a bipartite graph and an alternating path in it.



   A matching M is shown in bold, that is $M = \{\, \{1,5\}, \{2,6\}, \{3,8\} \,\}$. The path $(4, 6, 2, 5, 1, 8, 3, 9)$ is an alternating path because it has 7 edges, which is an odd number, the first vertex 4 and the last vertex 9 are non-covered, and the seven consecutive edges alternate between $E \setminus M$ and M: $\{4,6\} \notin M$, $\{6,2\} \in M$, $\{2,5\} \notin M$, etc.

   (a) **[14 marks]** Design an algorithm that—given a bipartite graph G and a matching M in it—finds an alternating path in G with respect to M, if there is one.

   What is the asymptotic running time of your algorithm? For full credit your algorithm should run in time $O(n + m)$.

   (b) **[12 marks]** Prove that if there is an alternating path in graph G with respect to a matching M then there is another matching in G of size $|M| + 1$.

   Illustrate this fact using the example above.

   (c) **[12 marks]** Prove that a matching in a bipartite graph G is not a matching of maximum size if and only if there exists an alternating path with respect to it.

   (d) **[12 marks]** Use the procedure from part (a) to design an efficient algorithm that—given a bipartite graph G—computes a maximum-size matching in G.

   Justify the correctness of your algorithm. What is the asymptotic running time of your algorithm? For full credit your algorithm should run in time $O(nm)$.

2. (a) **[5 marks]** Recall that a set of vertices $I \subseteq V$ in an *independent set* in an undirected graph $G = (V, E)$ if for every edge $\{v, w\} \in E$ we have $v \notin I$ or $w \notin I$.
   Consider a graph $G = (V, E)$ where $V = \{\, 1, 2, 3, 4, 5, 6, 7 \,\}$ and

   $$E = \{\, \{1,2\}, \{1,3\}, \{2,3\}, \{2,4\}, \{2,5\}, \{3,6\}, \{3,7\}, \{4,7\}, \{5,7\}, \{6,7\} \,\}.$$

   Draw the graph G and give an independent set of size 4 in G.

   (b) **[9 marks]** Describe a polynomial-time reduction from the 3SAT problem to the INDEPENDENT-SET decision problem and give a sketch of its correctness proof. Illustrate the reduction using an example with three clauses.

   (c) **[4 marks]** Let $S_1, S_2, \ldots, S_n$ be subsets of a set U. We say that k sets from the collection $S_1, S_2, \ldots, S_n$ are a *k-packing* if they are mutually disjoint.
   For example, consider $U = \{\, 1, 2, 3, 4, 5, 6, 7 \,\}$ and $S_1 = \{\, 3, 7 \,\}$, $S_2 = \{\, 2, 4 \,\}$, $S_3 = \{\, 3, 4, 5, 6 \,\}$, $S_4 = \{\, 5 \,\}$, $S_5 = \{\, 1 \,\}$, and $S_6 = \{\, 1, 2, 6, 7 \,\}$. Note that the two sets S_3 and S_5 are a 2-packing because $S_3 \cap S_5 = \emptyset$.
   Give a 4-packing for the above example.

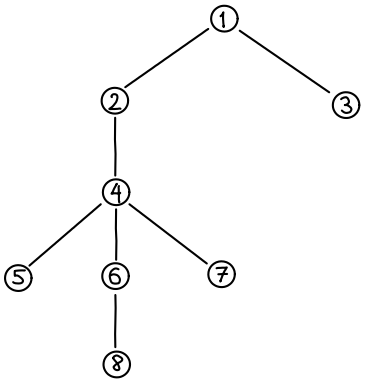
   (d) **[17 marks]** The INDEPENDENT-SET decision problem is—given an undirected graph $G = (V, E)$ and a number k as inputs—to decide if there is an independent set of size k.
   The SET-PACKING decision problem is—given a collection $S_1, S_2, \ldots, S_n$ of subsets of a set U and a number k as inputs—to decide if there is a k-packing.
   Give a polynomial-time reduction from the INDEPENDENT-SET problem to the SET-PACKING problem. Justify the correctness of your reduction.

   (e) **[7 marks]** Illustrate the execution of your polynomial-time reduction from part (d) when given the example graph from part (a) as the input.

   (f) **[8 marks]** Prove that the SET-PACKING decision problem is NP-complete.

3. Recall that a set $C \subseteq V$ is a *vertex cover* of an undirected graph $G = (V, E)$ if for every edge $\{v, w\} \in E$ we have $v \in C$ or $w \in C$.

   Consider trees that are represented by a parent array p, such that if a vertex v is the root then $p[v] = v$, and otherwise $p[v]$ is the parent of v. For example, the following tree T:



   is represented by the parent array:

Vertex v	1	2	3	4	5	6	7	8
Parent $p[v]$ of v	1	1	1	2	4	4	4	6

   (a) **[6 marks]** Argue that $\{ 2, 3, 5, 6, 7 \}$ is a vertex cover of the example tree T above. Give a vertex cover of T of size 4.

   (b) **[8 marks]** What is the minimum size of a vertex cover of the example tree T above?

   Is there a minimum-size vertex cover of tree T, which contains the root vertex 1? If yes, give such a vertex cover; if not, explain why.

   Is there a minimum-size vertex cover of tree T, which does not contain the root vertex 1? If yes, gives such a vertex cover; if not, explain why.

   (c) **[4 marks]** For a vertex v we let T_v be the subtree of tree T whose root is v.

   Draw subtrees T_8, T_6, T_4, and T_2 of the example tree T above.

   (d) **[10 marks]** Define $N(v)$ to be the minimum size of a vertex cover of subtree T_v that contains the root vertex v. Define $M(v)$ to be the minimum size of a vertex cover of subtree T_v.

   For the example tree T above, find values $N(v)$ and $M(v)$ for vertices $v = 8, 6, 4, 2$. In each case, give the minimum-size vertex covers of the relevant trees.

   (e) **[14 marks]** Describe an efficient polynomial-time algorithm that—given a tree as an input—computes the minimum size of a vertex cover of the tree.

   Justify the correctness of your algorithm. What is the asymptotic running time of your algorithm?

   (f) **[8 marks]** Design an linear-time algorithm that—given a rooted tree and the values $N(v)$ and $M(v)$ for all vertices v—computes a minimum-size vertex cover of the tree.

End (of CS 244; pages 6 – 9)